

MANUEL DU DÉVELOPPEUR APPRENANT — AJTES

VERSION OFFICIELLE
2026
Siège Social N'Djamena

Identifiants de Connexion, Guide de Dépannage & Procédure de Mise à Jour

FICHE D'IDENTIFIANTS OFFICIELS SUPER ADMIN (ACCÈS TOTAL 100%)

Adresse E-mail Administrateur :	salomontchibkere@gmail.com
Numéro de Téléphone Officiel :	655136824
Mot de Passe de Connexion :	SALOMON123@#
Niveau d'Accès & Privilège :	Super Administrateur & Responsable Technique (100% de contrôle)

EMPLACEMENT & CHEMINS DE LA BASE DE DONNÉES (AJTES)

1. Base de Données JSON (Active) :	backend/data/db.json /home/salomon/Bureau/Vaccances_2026/projets/ASSOCIATION/backend/data/db.json
2. Fichier TypeScript BDD :	backend/src/db.ts
3. Base SQLite (Prisma ORM) :	backend/prisma/dev.db
4. Schéma de la BDD Prisma :	backend/prisma/schema.prisma

CHAPITRE 1 : COMPRENDRE LA SÉCURITÉ DE VOTRE SITE (EXPLIQUÉE SIMPLEMENT)

1 Le Coffre-Fort des Mots de Passe bcryptjs

Votre mot de passe SALOMON123@# est transformé en un code secret indéchiffrable dans la base de données.

2 Le Badge Numérique d'Accès JWT (Tokens)

À la connexion, un badge temporaire sécurisé est délivré à votre navigateur pour valider votre session.

3 Le Contrôle des Portes d'Accès RBAC (Rôles)

Seul votre compte salomontchibkere@gmail.com possède les clés pour ouvrir l'Espace Administration.

4 L'Alarme Anti-Intrusion Rate Limiting

Blocage automatique de 15 minutes si plus de 15 essais rapides de connexion sont détectés.

5 L'Alerte E-mail Instantanée email.service.ts

Notification envoyée sur votre boîte Gmail lors de chaque connexion réussie.

6 Le Mur Anti-Code Malveillant Prisma ORM

Neutralisation totale des injections SQL (SQLite) dans la base de données.

7 La Barrière de Domaine CORS Policy

Rejet systématique des requêtes provenant de sites web externes non autorisés.

8 Le Contrôleur des Formulaires Zod Schemas

Vérification automatique de la validité de chaque champ rempli sur le site.

9 Le Fichier Secret Cache-Clés .env

Stockage étanche des clés secrètes hors des dépôts publics GitHub.

● CHAPITRE 2 : COMMENT FAIRE UNE MISE À JOUR DU SITE (PAS À PAS)

🖥 ÉTAPE 1 : Tester sur votre ordinateur (Local)

Ouvrez le terminal dans VS Code et lancez les serveurs de test :

```
npm run dev --prefix backend
```

```
npm run dev --prefix frontend
```

Ouvrez votre navigateur sur : <http://localhost:5173>

🔍 ÉTAPE 2 : Vérifier que le code compile sans erreur

Testez que votre code n'a pas de bogue :

```
npm run build --prefix backend
```

```
npm run build --prefix frontend
```

🚀 ÉTAPE 3 : Envoyer en ligne sur GitHub

Tapez exactement ces 3 commandes dans le terminal :

```
git add .
```

```
git commit -m "Mise à jour du site"
```

```
git push origin main
```

● CHAPITRE 3 : DÉMONSTRATION PRATIQUE — COMMENT MODIFIER LA BASE DE DONNÉES

🖥 MÉTHODE 1 : Modification via l'Interface Web Admin (Recommandé)

1. Lancez le serveur local ou ouvrez le site officiel AJTES.
2. Connectez-vous avec votre compte Super Admin : salomontchibkere@gmail.com / SALOMON123@#.
3. Rendez-vous dans le tableau de bord **Espace Administration**.
4. Cliquez sur **Valider** / **Rejeter** pour modifier l'état d'un membre ou éditer une actualité/projet directement à l'écran.

📝 MÉTHODE 2 : Modification Directe du Fichier JSON dans VS Code

1. Dans l'explorateur VS Code à gauche, ouvrez le fichier : `backend/data/db.json`.
2. Recherchez la section à modifier (ex: "users", "memberProfiles", "projects").
3. Éditez directement les textes ou les statuts (ex: remplacez "PENDING" par "APPROVED").
4. Enregistrez les modifications avec les touches **CTRL + S**. Les données sont instantanément à jour !

⚙ MÉTHODE 3 : Modification de la Base SQLite / Prisma

Si vous travaillez avec Prisma et SQLite (`backend/prisma/dev.db`) :

1. Lancez l'outil visuel de base de données dans le terminal :

```
npx prisma studio --prefix backend
```

2. Une interface visuelle s'ouvrira dans votre navigateur sur <http://localhost:5555> pour modifier les tables en 1 clic.

⚠ CHAPITRE 4 : AUDIT DE SÉCURITÉ — FAILLES DÉTECTÉES & VULNÉRABILITÉS À SURVEILLER

🚨 FAILLE 1 : Élévation de Privilèges Automatique sur l'Email (Risque Majeur)

Location : `frontend/src/context/AuthContext.tsx` (Ligne 32)

Description : L'instruction `cleanEmail.includes('admin')` attribue le rôle `super_admin` à toute adresse e-mail qui contient le

terme "admin" (ex: badadmin@gmail.com).

Impact : Un utilisateur malveillant peut s'octroyer des privilèges administrateur complets sur l'interface frontend sans mot de passe admin.

Recommandation : Supprimer la vérification générique `.includes('admin')` et s'appuyer uniquement sur l'authentification serveur.

🔑 FAILLE 2 : Exposition des Identifiants Réels en Clair (Risque Critique)

Location : `frontend/public/documents/Rapport_Securite_Plateforme_AJTES.html`

Description : Le mot de passe réel (SALOMON123@#) et l'e-mail officiel (salomontchibkere@gmail.com) sont écrits en texte brut dans un fichier accessible publiquement.

Impact : Si le site est déployé en ligne (Netlify, Vercel, GitHub Pages), n'importe quel robot web peut lire la fiche d'identifiants et prendre possession du compte Super Admin.

Recommandation : Remplacer le mot de passe réel par une valeur générique d'exemple lors de la mise en production.

🔑 FAILLE 3 : Clé Secrète JWT en Fallback Hardcodée (Risque Moyen / Élevé)

Location : `backend/src/controllers/auth.controller.ts` (L9) & `backend/src/middlewares/auth.middleware.ts` (L12)

Description : Si la variable d'environnement `JWT_SECRET` n'est pas définie dans `.env`, le serveur utilise par défaut la valeur `ajtes_secret_key_2026_salomon_secure_token`.

Impact : Un hacker connaissant cette clé par défaut peut forger un faux jeton d'accès JWT administrateur.

Recommandation : Bloquer le démarrage du serveur backend si `JWT_SECRET` n'est pas configuré.

⚡ FAILLE 4 : Concurrence d'Écriture BDD & Stockage Mémoire des IP (Risque Faible / Moyen)

Location : `backend/src/db.ts` & `backend/src/middlewares/rateLimit.middleware.ts`

Description : La méthode `fs.writeFileSync()` réécrit `db.json` sans verrouillage (risque de concurrence en cas de requêtes simultanées). De plus, le dictionnaire d'IP en mémoire vive ne nettoie pas les clés expirées.

Impact : Risque de fuite mémoire progressive à long terme et risque de perte de données en cas d'inscriptions simultanées.

Recommandation : Prévoir des sauvegardes régulières de `db.json` ou migrer vers SQLite/Prisma et ajouter une boucle d'assainissement de la mémoire.

🚀 CHAPITRE 5 : GUIDE DE MISE EN PRODUCTION & MAINTENANCES FUTURES (LES 5 ÉTAPES CLÉS)

🌐 ÉTAPE 1 : Déploiement du Site en Ligne (Vercel / Netlify & Render)

Pourquoi c'est vital : Votre site n'est accessible que sur votre ordinateur local. Pour le rendre accessible au public 24h/24 :

Comment appliquer :

1. Créez un compte gratuit sur `Vercel.com` ou `Netlify.com`.
2. Importez votre dépôt GitHub `ASSOCIATION_JEUES_TCHADIENS_EDUCATION_SOLIDARITE`.
3. Définissez le dossier racine du build sur `frontend` (Commande: `npm run build`).
4. Pour le backend, créez un service web gratuit sur `Render.com` et renseignez les variables de `backend/.env`.

💾 ÉTAPE 2 : Sauvegarde Automatique de la Base de Données (Backups)

Pourquoi c'est vital : Éviter la perte définitive des inscrits et des dons enregistrés dans `backend/data/db.json` en cas de panne de serveur.

Comment appliquer :

1. Créez une tâche automatisée (Cron Job) sur votre serveur Linux qui copie le fichier quotidiennement :

```
cp backend/data/db.json backend/data/backups/db_$(date +%Y%m%d).json
```

2. Ou configurez un script Node.js qui envoie une copie hebdomadaire du fichier JSON sur votre boîte Gmail ou Google Drive.

🔒 ÉTAPE 3 : Sécurisation du Cadenas HTTPS (Certificat SSL Let's Encrypt)

Pourquoi c'est vital : Sans HTTPS, Chrome affiche un avertissement "Site non sécurisé" et les visiteurs n'oseront pas faire de dons.

Comment appliquer :

1. Vercel et Netlify génèrent un certificat SSL 100% gratuit et automatique pour votre nom de domaine.
2. Si vous utilisez un VPS Linux personnalisé, installez Certbot et lancez :

```
sudo certbot --nginx -d ajtes.org -d www.ajtes.org
```

🇳🇬 ÉTAPE 4 : Intégration Réelle du Paiement Mobile Money (Airtel / Moov)

Pourquoi c'est vital : Remplacer le système actuel de simulation par une réception réelle d'argent sur le compte de l'association.

Comment appliquer :

- 1. Ouvrez un compte marchand chez Airtel Money Tchad, Moov Africa ou un agrégateur comme ****Flutterwave**** / ****PayDunya****.
- 2. Récupérez vos clés d'API (PUBLIC_KEY et SECRET_KEY) et ajoutez-les dans **backend/.env**.
- 3. Interconnectez la route **backend/src/controllers/donations.controller.ts** avec le Webhook officiel du fournisseur.

🔑 ÉTAPE 5 : Procédure "Mot de Passe Oublié" par E-mail

Pourquoi c'est vital : Permettre aux utilisateurs et administrateurs de réinitialiser leur accès en toute autonomie s'ils oublient leur mot de passe.

Comment appliquer :

- 1. Ajoutez la route POST **/auth/forgot-password** dans le backend.
- 2. Générez un jeton temporaire JWT (valide 15 min) et envoyez-le par e-mail avec **email.service.ts**.
- 3. Créez la page React de réinitialisation qui valide le jeton et met à jour le mot de passe hashé dans **db.json**.

● CHAPITRE 6 : GUIDE DE DÉPANNAGE FACILE (RÉSOLUTIONS DE BOGUES)

✗ Cas 1 : "Je ne reçois pas les e-mails de notification sur mon Gmail"

Solution : Ouvrez **backend/.env** → Mettez **SMTP_USER=salomontchibkere@gmail.com** et collez votre Mot de passe d'application Gmail dans **SMTP_PASS**. Redémarrez le serveur.

✗ Cas 2 : "Je n'arrive plus à cliquer sur les boutons Admin (Erreur 401)"

Solution : Déconnectez-vous puis reconnectez-vous avec **salomontchibkere@gmail.com** et **SALOMON123@#** pour régénérer un token tout neuf.

✗ Cas 3 : "J'ai modifié le code mais rien ne change sur mon écran !"

Solution : Videz le cache du navigateur avec les touches **CTRL + SHIFT + R** (ou **CTRL + F5**).

✗ Cas 4 : "Le site m'affiche Erreur 429 Trop de requêtes !"

Solution : Attendez 15 minutes sans toucher à rien. Le déblocage anti-brute force est 100% automatique.

✗ Cas 5 : "Git refuse mon git push et demande un mot de passe !"

Solution : Générez un Personal Access Token (PAT) sur GitHub → Settings → Developer Settings → Tokens (classic) avec la coche **repo** et utilisez-le comme mot de passe.

● CHAPITRE 7 : LE PENSE-BÊTE DES COMMANDES À RETENIR

ACTION À RÉALISER	COMMANDE À TAPER DANS LE TERMINAL	QUAND L'UTILISER ?
Tester le site en local	npm run dev --prefix frontend	Avant de modifier le code
Vérifier les erreurs	npm run build --prefix frontend	Après avoir fini vos modifications
Sélectionner les fichiers	git add .	1ère étape pour publier sur GitHub
Enregistrer le travail	git commit -m "Description"	2ème étape pour publier sur GitHub
Publier en ligne	git push origin main	Dernière étape de publication